

Python Topic 18 — Async Programming (async / await)

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert • The heart of FastAPI

Layer 1 — New to Python

Async lets your program do other work while waiting for slow things (network calls, file reads, database queries) instead of sitting idle.

Imagine a waiter. A **synchronous** waiter takes your order, stands at the kitchen until the food is ready, *then* serves the next table. An **async** waiter takes your order, sends it to the kitchen, and while it cooks, takes other tables' orders. Same waiter, far more efficient — because most of the time was just *waiting*.

```
import asyncio

async def make_coffee():          # "async def" defines an async function
    print("Start brewing")
    await asyncio.sleep(2)        # "await" = wait here, but let others run
    print("Coffee ready")

asyncio.run(make_coffee())        # run the async function
```

Two new keywords: **async def** (defines an async function) and **await** (pause here without blocking everything else).

Layer 2 — Intermediate (real code + how it works)

You already know this from JavaScript

```
// JS
async function getData() {
    const res = await fetch(url);
    return await res.json();
}

# Python — same async/await concept
async def get_data():
    async with aiohttp.ClientSession() as session:
        async with session.get(url) as res:
            return await res.json()
```

The key win — running things concurrently

```
import asyncio

async def fetch(name, seconds):
    print(f"{name} started")
    await asyncio.sleep(seconds)    # simulate a slow API call
    print(f"{name} done")
    return name

# ❌ Sequential – one after another (total: 3 seconds)
async def slow():
    await fetch("A", 1)
    await fetch("B", 1)
    await fetch("C", 1)

# ✅ Concurrent – all at once (total: ~1 second!)
async def fast():
    await asyncio.gather(
        fetch("A", 1),
        fetch("B", 1),
        fetch("C", 1),
    )
```

i `asyncio.gather()` runs multiple async tasks concurrently and waits for all — the equivalent of JS `Promise.all()`. Three 1-second calls finish in ~1 second because they wait *together*.

A coroutine doesn't run until awaited

```
async def greet():
    return "Hi"

coro = greet()    # ⚠️ does NOT run – just creates a coroutine object
result = await greet()    # runs it → "Hi" (inside an async function)
result = asyncio.run(greet())    # or at top level
```

⚠️ **#1 beginner mistake:** calling an async function returns a **coroutine** (like a JS promise) — it only executes when awaited or run. Forgetting to `await` gives a "coroutine never awaited" warning.

Layer 3 — Expert (internals & gotchas)

1. Async only helps with I/O-bound work, not CPU-bound

```
#  GOOD use — waiting on I/O (network, disk, DB)
await fetch_from_api()
await read_database()

#  WRONG — CPU-heavy work blocks the event loop anyway
async def crunch():
    result = sum(i**2 for i in range(10**8))    # no await — blocks everything!
```

Async shines when you're **waiting** (network/disk/DB). For CPU-heavy work (number crunching, image processing), async doesn't help — use `multiprocessing` or threads. A key distinction interviewers probe.

2. The event loop — the engine underneath

Async runs on a single-threaded **event loop** that juggles tasks. When a task hits `await`, it hands control back to the loop, which runs other ready tasks until the awaited thing completes. One thread, many tasks interleaved — same model as JavaScript's event loop.

3. Never mix blocking calls into async code

```
import time, asyncio

async def bad():
    time.sleep(2)          #  BLOCKS the whole event loop — freezes everything!

async def good():
    await asyncio.sleep(2) #  non-blocking — yields to the loop
```

 **Critical:** a blocking call (`time.sleep` , `requests.get` , a sync DB driver) inside async code freezes the **entire** event loop — all concurrency dies. Use async libraries (`aiohttp` not `requests` , `asyncpg` not sync `psycopg2`) or offload with `asyncio.to_thread()` .

4. `asyncio.create_task()` — fire off work without waiting immediately

```
async def main():
    task = asyncio.create_task(fetch("A", 2))    # starts running NOW
    # do other work here while it runs...
    await task                                  # await the result later
```

`create_task` schedules a coroutine to run in the background immediately; you await it when you need the result. `gather` is built on this.

5. `async for` and `async with`

```

async for item in async_generator():    # async iteration (e.g. streaming DB rows)
    process(item)

async with aiohttp.ClientSession() as session:    # async context manager
    ...

```

The async versions of `for` and `with` — used with async generators and resources. Combine `async` generators with Topic 17 to stream LLM tokens asynchronously.

6. The FastAPI / AI connection

```

from fastapi import FastAPI
app = FastAPI()

@app.get("/data")
async def get_data():                # async route – handles many requests concurrently
    result = await call_llm(prompt)  # await the LLM without blocking other requests
    return result

```

✅ **Heart of your backend work:** FastAPI is async-first — while one request waits on an LLM/DB call, the server handles others. For MSG2AI handling many concurrent WhatsApp messages, async lets one server juggle thousands of waiting requests efficiently.

7. `asyncio.run()` only at the top level

```

# ✅ one asyncio.run() at the entry point
asyncio.run(main())

# ❌ don't call asyncio.run() inside an already-running loop (FastAPI, Jupyter)
# → RuntimeError: event loop is already running

```

Inside an already-async context (FastAPI handlers, Jupyter), just `await` directly — don't call `asyncio.run()` again.

🎯 Interview Q&A Cards

Q: When does async actually help?

A: For I/O-bound work — waiting on network, disk, or DB. It does **not** speed up CPU-bound work (that needs multiprocessing/threads), since CPU tasks block the single-threaded event loop.

Q: What does calling an async function return?

A: A coroutine object — it doesn't run until you `await` it or pass it to `asyncio.run` / `gather`. Forgetting to `await` is the classic bug.

Q: What breaks if you put a blocking call in async code?

A: It freezes the entire event loop — all concurrent tasks stall. Use async-native libraries or offload blocking work with `asyncio.to_thread()`.

Q: `asyncio.gather` — what's the JS equivalent?

A: `Promise.all()`. It runs multiple coroutines concurrently and waits for all to finish, so their wait times overlap instead of adding up.

 **Memory hook:** `async def` + `await` = do other work while waiting. Event loop, single thread, like JS.
`gather` = `Promise.all`. Only for I/O, not CPU. Never block the loop.

✓ **One-liner for interviews:** "`async def` / `await` enable concurrency on a single-threaded event loop — ideal for I/O-bound work (network, DB), not CPU-bound. Calling an async function returns a coroutine that runs only when awaited. `asyncio.gather` runs tasks concurrently (like `Promise.all`). Never put blocking calls in async code — they freeze the event loop. FastAPI is async-first for handling many concurrent requests."